WordPress Operations Runbook

March 2026

Contents

1 WordPress Operations Runbook **3**
1.1 Emergency Quick-Reference Card . . . 3
1.2 Section 1: Document Information . . . 3
1.2.1 1.1 Purpose . . . 3
1.2.2 1.2 Audience . . . 4
1.2.3 1.3 Conventions . . . 4
1.2.4 1.4 Version History . . . 4
1.3 Section 2: Site Overview . . . 5
1.3.1 2.1 Site Identity . . . 5
1.3.2 2.2 Credentials and Secrets . . . 5
1.3.3 2.3 User Roles and Responsibilities . . . 6
1.4 Section 3: Environment Reference . . . 6
1.4.1 3.1 Architecture Overview: LEMP Stack . . . 6
1.4.2 3.2 Service Configuration Reference . . . 7
1.4.3 3.3 Caching Architecture . . . 7
1.4.4 3.4 File System Locations . . . 8
1.4.5 3.5 SSL/TLS Configuration . . . 8
1.5 Section 4: Development Workflow . . . 9
1.5.1 4.1 Version Control Strategy . . . 9
1.5.2 4.2 Deployment Workflow . . . 10
1.5.3 4.3 Staging Synchronization . . . 12
1.6 Section 5: Security Hardening . . . 13
1.6.1 5.1 Firewall and SSH Configuration . . . 13
1.6.2 5.2 WordPress Hardening . . . 14
1.6.3 5.3 HTTP Security Headers . . . 15
1.6.4 5.4 REST API and XML-RPC Management . . . 16
1.6.5 5.5 File Integrity Monitoring (FIM) . . . 16
1.7 Section 6: Routine Maintenance . . . 17
1.7.1 6.1 Maintenance Calendar . . . 17
1.7.2 6.2 WordPress Core Updates . . . 18
1.7.3 6.3 Plugin and Theme Updates . . . 19
1.7.4 6.4 Database Optimization . . . 20
1.7.5 6.5 PHP Version Upgrade . . . 21
1.7.6 6.6 WordPress Cron (WP-Cron) Management . . . 23
1.7.7 6.7 Transactional Email Management . . . 23
1.7.8 6.8 Log Rotation . . . 24
1.7.9 6.9 Monitoring and Alerting . . . 26
1.8 Section 7: Backup Procedures . . . 27
1.8.1 7.1 Backup Strategy . . . 27
1.8.2 7.2 Automated Backup Script . . . 28
1.8.3 7.3 Backup Verification . . . 30
1.9 Section 8: Deployment Procedures . . . 31
1.9.1 8.1 Code Deployment . . . 31
1.9.2 8.2 Database Migration . . . 31
1.9.3 8.3 Rollback Procedure . . . 33
1.10 Section 9: Content Operations . . . 34
1.10.1 9.1 Media Management . . . 34

1.10.2 9.2 Publishing Workflow . . . 35
1.10.3 9.3 Taxonomy and Permalink Management . . . 36
1.10.4 9.4 Search and Indexing . . . 37
1.11 Section 10: Incident Response . . . 38
1.11.1 10.1 Severity Classification . . . 38
1.11.2 10.2 Site Down / 500 Error Triage . . . 39
1.11.3 10.3 Security Breach Response . . . 41
1.11.4 10.4 Escalation Path . . . 43
1.11.5 10.5 Performance Degradation . . . 44
1.11.6 10.6 Post-Incident Review . . . 45
1.12 Section 11: Disaster Recovery . . . 46
1.12.1 11.1 Recovery Objectives . . . 46
1.12.2 11.2 Full Site Restore from Backup . . . 47
1.12.3 11.3 Database-Only Recovery . . . 49
1.12.4 11.4 Server Migration . . . 51
1.13 Appendix A: File Permissions Reference . . . 52
1.13.1 A.1 WordPress File Permissions Table . . . 52
1.13.2 A.2 Permission Reset Script . . . 53
1.14 Appendix B: wp-config.php Key Settings . . . 55
1.14.1 B.1 Database Configuration . . . 55
1.14.2 B.2 Security Keys and Salts . . . 55
1.14.3 B.3 Security Constants . . . 55
1.14.4 B.4 Debugging Constants . . . 56
1.14.5 B.5 Performance Constants . . . 56
1.14.6 B.6 WordPress Cron Constants . . . 57
1.14.7 B.7 Multisite Configuration (if applicable) . . . 57
1.14.8 B.8 Path Configuration (Optional) . . . 57
1.14.9 B.9 Complete Recommended wp-config.php Template . . . 58
1.15 Appendix C: Change Log . . . 60
1.15.1 Release History . . . 60
1.16 Quick Reference Card (Printable) . . . 60
1.17 Related Documentation . . . 61
1.18 Glossary . . . 62
1.19 Document Metadata . . . 62

1 WordPress Operations Runbook

Version: 2.0
Last Updated: February 2026
Next Review: February 2027
Owner: [CUSTOMIZE: Operations Team Name]
Environment: [CUSTOMIZE: Production/Staging/Development]

1.1 Emergency Quick-Reference Card

Use this table to quickly find solutions to common issues.

Symptom	Go To	First Command
Site returns 500 error	Section 10.2	`wp plugin status`
Site is slow/not responding	Section 10.5	`top` and `free -h`
Missing database tables	Section 8.3	`wp db check`
Plugin causing crashes	Section 10.2	`wp plugin deactivate --all`
Need to restore from backup	Section 11.2	Check backup integrity first
WordPress cannot write to disk	Appendix A	`wp server list --format=csv`
SSL certificate expired	Section 3.5	`openssl x509 -in [cert] -noout -text`
Users locked out	Section 5.3	`wp user list --format=table`
Hacked/malware suspected	Section 10.3	Isolate site immediately, see escalation
Database won't start	Section 11.2	`systemctl status mysql`

1.2 Section 1: Document Information

1.2.1 1.1 Purpose

This runbook provides comprehensive operational guidance for managing, maintaining, securing, and troubleshooting a WordPress installation in production. It covers:

- Daily maintenance and monitoring

- Emergency response procedures
- Backup and disaster recovery
- Security hardening and incident response
- Deployment workflows and rollback procedures

1.2.2 1.2 Audience

This document is intended for:

- WordPress system administrators
- DevOps engineers
- Site reliability engineers (SREs)
- Technical support staff
- Security operations center (SOC) personnel

1.2.3 1.3 Conventions

Convention	Meaning
`code`	Commands, file paths, or configuration values
`[CUSTOMIZE: ...]`	Site-specific values to be filled in
> **WARNING:**	Critical safety information
> **NOTE:**	Important context or clarification
> **CUSTOMIZE:**	Settings specific to your environment
Expected:	Anticipated output or result
italics	Emphasis or variable placeholders

1.2.4 1.4 Version History

Version	Date	Changes	Author
2.0	Feb 2026	Comprehensive update with security hardening sections	[CUSTOMIZE]
1.5	Aug 2025	Added disaster recovery procedures	[CUSTOMIZE]
1.0	Feb 2025	Initial release	[CUSTOMIZE]

1.3 Section 2: Site Overview

1.3.1 2.1 Site Identity

Property	Value
Domain	[CUSTOMIZE: example.com]
Site Name	[CUSTOMIZE: My WordPress Site]
WordPress Version	[CUSTOMIZE: 6.4+]
PHP Version	[CUSTOMIZE: 8.1+]
Server OS	[CUSTOMIZE: Ubuntu 22.04 LTS]
Hosting Type	[CUSTOMIZE: Self-hosted/Managed/VPS]
Server Hostname	[CUSTOMIZE: wp-prod-01.example.com]
IP Address	[CUSTOMIZE: 192.168.1.100]
Database Engine	[CUSTOMIZE: MySQL 8.0 / MariaDB 10.6]

1.3.2 2.2 Credentials and Secrets

> **WARNING:** Never commit credentials to version control. Store all secrets in a secure vault (1Password, Vault, etc.).

Credential Type	Storage Location	Rotation Schedule	Owner
Database credentials	[CUSTOMIZE: Secure vault]	Every 90 days	[CUSTOMIZE]
SFTP/SSH keys	[CUSTOMIZE: Secure vault]	Every 180 days	[CUSTOMIZE]
WP-CLI OAuth token	[CUSTOMIZE: ~/.wp-cli/config.yml]	Every 180 days	[CUSTOMIZE]
Third-party API keys	[CUSTOMIZE: wp-config.php env vars]	Every 180 days	[CUSTOMIZE]
SSL certificates	[CUSTOMIZE: /etc/letsencrypt]	Auto-renew 30 days before expiry	[CUSTOMIZE]

1.3.3 2.3 User Roles and Responsibilities

Role	Responsibilities	On-Call	Contact
System Administrator	Server infrastructure, backups, security	Yes	[CUSTOMIZE]
WordPress Administrator	Plugin management, user access, content	Yes	[CUSTOMIZE]
Database Administrator	Database maintenance, optimization, recovery	On-rotation	[CUSTOMIZE]
Security Officer	Vulnerability patching, security audits	On-rotation	[CUSTOMIZE]
Content Manager	Publishing, editorial workflow, media	Business hours	[CUSTOMIZE]

1.4 Section 3: Environment Reference

1.4.1 3.1 Architecture Overview: LEMP Stack

The WordPress site runs on a LEMP (Linux, Nginx, MySQL, PHP) stack:

```
┌──────────────────────────────────────────────────────────┐
│                        Client Browsers                   │
└──────────────────────────┬───────────────────────────────┘
                           │ HTTPS
                           ▼
┌──────────────────────────────────────────────────────────┐
│              Nginx (Reverse Proxy + Web Server)          │
│           (Port 80 → 443, SSL Termination)               │
└──────────────────────────┬───────────────────────────────┘
                           │ FastCGI
                           ▼
┌──────────────────────────────────────────────────────────┐
│                    PHP-FPM (Application)                 │
│         (WordPress, WP-CLI, Plugins, Themes)             │
```

```
└──────────────────────────┬──────────────────────────────────────┘
                           │
                           │ TCP/Socket
                           ▼
┌──────────────────────────────────────────────────────────────────┐
│              MySQL / MariaDB (Database)                          │
│         (WordPress Posts, Users, Metadata)                       │
└──────────────────────────────────────────────────────────────────┘


                    Caching Layer (Optional)
┌──────────────────────────────────────────────────────────────────┐
│  Redis (Object Cache) / Memcached (Session Store)                │
└──────────────────────────────────────────────────────────────────┘
```

1.4.2 3.2 Service Configuration Reference

Service	Version	Port	User	Config Path	Log Path
Nginx	[CUSTOMIZE: 1.24+]	80, 443	`www-data`	`/etc/nginx/nginx.conf`	`/var/log/nginx/`
PHP-FPM	[CUSTOMIZE: 8.1+]	9000 (socket)	`www-data`	`/etc/php/8.1/fpm/p`	`/var/log/php*.log`
MySQL	[CUSTOMIZE: 8.0+]	3306 (local)	`mysql`	`/etc/mysql/mysql.conf`	`/var/log/mysql/error.log`
Redis	[CUSTOMIZE: 7.0+]	6379	`redis`	`/etc/redis/redis.c`	`/var/log/redis/redis-server.log`

1.4.3 3.3 Caching Architecture

Layer	Technology	Purpose	TTL
Page Cache	[CUSTOMIZE: W3 Total Cache / WP Super Cache / Litespeed]	Full page HTML	1 hour
Object Cache	[CUSTOMIZE: Redis / Memcached / Object Cache Pro]	Database queries, transients	24 hours
Browser Cache	HTTP headers (Cache-Control)	Client-side assets	1 week (CSS/JS), 1 month (images)

Layer	Technology	Purpose	TTL
Database Query Cache	MySQL Query Cache	SELECT statements	Disabled / configured
CDN Cache	[CUSTOMIZE: Cloudflare / Bunny / AWS CloudFront]	Static assets, images	Varies

1.4.4 3.4 File System Locations

```
/home/wordpress/
├── public_html/                    # WordPress root
│   ├── wp-admin/                   # WordPress admin
│   ├── wp-content/
│   │   ├── plugins/                # Custom and third-party plugins
│   │   ├── themes/                 # Active and inactive themes
│   │   ├── uploads/                # User-uploaded media
│   │   │   ├── [YEAR]/[MONTH]/    # Organized by date
│   │   │   └── cache/              # Cache plugin files
│   │   └── mu-plugins/             # Must-use plugins
│   ├── wp-includes/                # WordPress core files
│   ├── wp-config.php               # Core configuration (SECRET)
│   ├── .htaccess                   # Rewrite rules (if using Apache)
│   ├── index.php                   # WordPress index
│   └── readme.html                 # WordPress info (REMOVE IN PRODUCTION)
├── backup/                         # Local backup directory
│   ├── daily/
│   ├── weekly/
│   └── monthly/
├── logs/                           # Application logs
│   ├── access.log
│   ├── error.log
│   └── php-errors.log
└── ssl/                            # SSL certificates (optional)
    ├── cert.pem
    └── key.pem
```

1.4.5 3.5 SSL/TLS Configuration

> **CUSTOMIZE:** Update paths and domains based on your certificate provider.

Certificate Details:

Property	Value
Provider	[CUSTOMIZE: Let's Encrypt / Paid CA]
Certificate Path	[CUSTOMIZE: /etc/letsencrypt/live/example.com/]
Key Path	[CUSTOMIZE: /etc/letsencrypt/live/example.com/privkey.pem]
Certificate Chain	[CUSTOMIZE: /etc/letsencrypt/live/example.com/fullchain.pem]
Renewal Method	[CUSTOMIZE: Certbot cron / Manual renewal]
Renewal Schedule	30 days before expiry
Current Expiration	[CUSTOMIZE: YYYY-MM-DD]

Verify SSL Certificate:

```
# Check certificate details
openssl x509 -in /etc/letsencrypt/live/[CUSTOMIZE: example.com]/cert.pem -noout -text

# Check certificate expiration
openssl x509 -in /etc/letsencrypt/live/[CUSTOMIZE: example.com]/cert.pem -noout -dates

# Test SSL configuration
openssl s_client -connect [CUSTOMIZE: example.com]:443 -tls1_2

# Verify from Nginx
curl -vI https://[CUSTOMIZE: example.com]
```

HSTS (HTTP Strict Transport Security):

The following header is configured in Nginx to enforce HTTPS:

```
add_header Strict-Transport-Security "max-age=31536000; includeSubDomains; preload"
↪ always;
```

This tells browsers to only connect via HTTPS for one year.

1.5 Section 4: Development Workflow

1.5.1 4.1 Version Control Strategy

Repository Structure:

```
wordpress-repo/
├── wp-content/
│   ├── plugins/            # Custom plugins
│   ├── themes/             # Custom themes
│   └── .gitignore
├── wp-config.php           # Template (no secrets)
├── .htaccess              # Rewrite rules
├── README.md              # Documentation
└── .git
```

> **WARNING:** Never commit to production directly. Always use the staging → production workflow.

Branches:

- `main` - Production code (tagged releases)
- `develop` - Integration branch for features
- `hotfix/` - Emergency fixes for production
- `feature/` - New features for next release

1.5.2 4.2 Deployment Workflow

Time Estimate: 15-45 minutes depending on scope

Prerequisites: - All code reviewed and tested - Database backups current - Staging environment synchronized - Change window approved

Steps:

1. **Prepare Release Branch**

   ```
   git checkout -b release/version-X.Y.Z develop
   git tag -a vX.Y.Z -m "Release version X.Y.Z"
   ```

2. **Deploy to Staging First**

   ```
   # SSH to staging server
   ssh [CUSTOMIZE: staging-user@staging.example.com]
   cd /home/wordpress/public_html

   # Pull latest code
   git fetch origin
   git checkout release/version-X.Y.Z

   # Install/update dependencies
   composer update --no-dev
   wp plugin update --all
   ```

```
wp theme update --all

# Run database migrations if needed
wp db query < migrations/pending-migration.sql
```

3. **Test on Staging**
   - Verify site loads: `curl -I https://[CUSTOMIZE: staging.example.com]`
   - Check critical pages for functionality
   - Run automated tests: `npm run test`
   - Verify no console errors
   - Test user registration/login
   - Test form submissions
4. **Approve for Production**
   - Notify team lead
   - Get sign-off from project manager
   - Document any issues found
5. **Deploy to Production**

```
# SSH to production server
ssh [CUSTOMIZE: prod-user@prod.example.com]
cd /home/wordpress/public_html

# Create backup before deploy
wp db export backups/pre-deploy-$(date +%Y%m%d-%H%M%S).sql

# Pull latest code
git fetch origin
git checkout release/version-X.Y.Z

# Install dependencies
composer update --no-dev

# Update plugins (one by one, monitor after each)
wp plugin update wp-plugin-name-1
wp plugin update wp-plugin-name-2

# Update themes if changed
wp theme update custom-theme
```

6. **Post-Deployment Verification**

```
# Verify WordPress is healthy
wp core is-installed && echo "WordPress OK"
```

```
# Check for PHP errors in logs
tail -20 /var/log/php-errors.log

# Verify database connection
wp db check

# Test site responsiveness
curl -s -w "%{http_code}\n" -o /dev/null https://[CUSTOMIZE: example.com]
```

7. **Monitor for Issues**
   - Watch error logs for 15 minutes
   - Monitor site uptime and response time
   - Check critical user flows
   - Be ready to rollback if needed

1.5.3 4.3 Staging Synchronization

Sync Staging Environment from Production

Time Estimate: 10-30 minutes

> **WARNING:** Staging sync will overwrite staging data. Notify team before starting.

1. **Backup Staging Database (in case of error)**

```
ssh [CUSTOMIZE: staging-user@staging.example.com]
cd /home/wordpress
wp db export backup/staging-pre-sync-$(date +%Y%m%d-%H%M%S).sql
```

2. **Export Production Database**

```
ssh [CUSTOMIZE: prod-user@prod.example.com]
cd /home/wordpress
wp db export - --single-transaction > /tmp/prod-dump.sql
```

3. **Transfer to Staging**

```
# From staging server
scp [CUSTOMIZE: prod-user@prod.example.com]:/tmp/prod-dump.sql ./prod-dump.sql
```

4. **Import to Staging**

```
# On staging server
wp db import prod-dump.sql

# Update WordPress URLs (critical for staging)
wp search-replace "https://example.com" "https://staging.example.com" --all-tables
```

5. **Sync Code**

```
# On staging server
git fetch origin
git checkout origin/main
wp plugin list
```

6. **Clear Caches**

```
wp cache flush
wp w3-total-cache flush all
wp redis flush-db
```

1.6 Section 5: Security Hardening

1.6.1 5.1 Firewall and SSH Configuration

UFW (Uncomplicated Firewall) Setup:

```
# Enable firewall
sudo ufw enable

# Allow SSH (critical - do this FIRST)
sudo ufw allow 22/tcp

# Allow HTTP and HTTPS
sudo ufw allow 80/tcp
sudo ufw allow 443/tcp

# Block all other incoming traffic
sudo ufw default deny incoming
sudo ufw default allow outgoing

# Verify rules
sudo ufw status
```

Expected: SSH, HTTP, HTTPS allowed; all other ports denied.

SSH Hardening (in `/etc/ssh/sshd_config`):

```
# Disable root login
PermitRootLogin no

# Disable password authentication (use keys only)
PasswordAuthentication no
PubkeyAuthentication yes

# Change default port (optional - use non-standard port)
```

```
Port [CUSTOMIZE: 2222]

# Limit concurrent sessions
MaxStartups 10:30:100

# Set login grace time
LoginGraceTime 30s

# Enable X11 forwarding only if needed
X11Forwarding no

# Restrict users who can SSH
AllowUsers [CUSTOMIZE: admin@*.example.com wordpress@*.example.com]
```

Restart SSH:

```
sudo systemctl restart sshd
```

> **WARNING:** Test SSH access before closing your current session to avoid lockout.

1.6.2 5.2 WordPress Hardening

Remove Unnecessary Files:

```
# Remove installation files
rm /home/wordpress/public_html/wp-admin/install.php

# Remove readme files
rm /home/wordpress/public_html/readme.html
rm /home/wordpress/public_html/wp-admin/readme.html

# Remove license files
rm /home/wordpress/public_html/license.txt
```

Disable File Editing:

Add to `wp-config.php`:

```
// Disable theme and plugin file editor (CRITICAL for security)
define('DISALLOW_FILE_MODS', true);
```

> **WARNING:** When `DISALLOW_FILE_MODS` is enabled, users cannot edit files through the WordPress admin panel. Updates must go through deployment.

Limit XML-RPC and REST API:

```
# Disable XML-RPC (legacy remote publishing protocol)
wp post-type update post --rest_enabled=true  # For REST API
```

```
# If not needed, disable entirely (add to wp-config.php):
# define('XMLRPC_REQUEST', false);

# Disable REST API for non-authenticated users (if not needed)
# Add to functions.php of theme or plugin:
# add_filter('rest_authentication_errors', function($result) {
#     if (!is_user_logged_in() && !current_user_can('read')) {
#         return new WP_Error('rest_not_permitted', 'REST API disabled for
↪  non-authenticated users', ['status' => 401]);
#     }
#     return $result;
# });
```

1.6.3 5.3 HTTP Security Headers

Configure in Nginx (`/etc/nginx/conf.d/security-headers.conf`):

```
# Prevent MIME type sniffing
add_header X-Content-Type-Options "nosniff" always;

# Enable XSS protection in older browsers
add_header X-XSS-Protection "1; mode=block" always;

# Clickjacking protection - prevent embedding in frames
add_header X-Frame-Options "SAMEORIGIN" always;

# Referrer Policy
add_header Referrer-Policy "strict-origin-when-cross-origin" always;

# Permissions Policy (formerly Feature Policy) - limit browser features
add_header Permissions-Policy "geolocation=(), microphone=(), camera=()" always;

# Content Security Policy (CSP) - restrict content sources
add_header Content-Security-Policy "default-src 'self'; script-src 'self'
↪  'unsafe-inline' 'unsafe-eval' cdn.example.com; style-src 'self' 'unsafe-inline';
↪  img-src 'self' data: https:; font-src 'self' data:;" always;

# HSTS - Force HTTPS
add_header Strict-Transport-Security "max-age=31536000; includeSubDomains; preload"
↪  always;
```

Reload Nginx:

```
sudo nginx -t  # Test configuration
sudo systemctl reload nginx
```

Verify Headers:

```
curl -I https://[CUSTOMIZE: example.com] | grep -i "X-\|Strict\|Content-Security"
```

1.6.4 5.4 REST API and XML-RPC Management

Disable XML-RPC (recommended):

Option 1: Add to `wp-config.php`:

```
define('XMLRPC_REQUEST', false);
```

Option 2: Block in Nginx `/etc/nginx/conf.d/block-xmlrpc.conf`:

```
location /xmlrpc.php {
    deny all;
    access_log off;
    log_not_found off;
}
```

Control REST API Access:

```
# Check if REST API is enabled
wp rest-api get-routes

# Disable REST API for non-authenticated users (if not needed)
# This requires custom code in your theme or plugin
```

Add to theme's `functions.php`:

```
// Require authentication for REST API (if not needed publicly)
add_filter('rest_authentication_errors', function($result) {
    if (!is_user_logged_in()) {
        return new WP_Error('rest_not_authenticated', 'REST API requires
        ↪  authentication', ['status' => 401]);
    }
    return $result;
});
```

Monitor REST API Usage:

```
# Check Nginx logs for REST API calls
grep "wp-json" /var/log/nginx/access.log | tail -20
```

1.6.5 5.5 File Integrity Monitoring (FIM)

Setup AIDE (Advanced Intrusion Detection Environment):

```
# Install AIDE
sudo apt-get install aide aide-common
```

```
# Initialize AIDE database (takes several minutes)
sudo aideinit

# Move database to production location
sudo mv /var/lib/aide/aide.db /var/lib/aide/aide.db.orig
sudo cp /var/lib/aide/aide.db.new /var/lib/aide/aide.db

# Check file integrity
sudo aide --check

# Update database after planned changes
sudo aide --init
sudo mv /var/lib/aide/aide.db.new /var/lib/aide/aide.db
```

Configure Automated Checking:

Add to crontab (`crontab -e`):

```
# Run AIDE check daily at 2 AM
0 2 * * * /usr/bin/aide --check | mail -s "AIDE Report for $(hostname)" root@example.com
```

Monitor WordPress File Changes:

```
# List recently modified WordPress files (potential hack indicator)
find /home/wordpress/public_html -type f -mtime -1 -ls

# Check for suspicious file permissions
find /home/wordpress/public_html -type f -perm /022 -ls

# Find recently added PHP files
find /home/wordpress/public_html -name "*.php" -type f -mtime -7
```

1.7 Section 6: Routine Maintenance

1.7.1 6.1 Maintenance Calendar

Task	Frequency	Duration	Owner	Window
Monitor error logs	Daily	5 min	Sysadmin	Any time
Check plugin/theme updates	Daily	10 min	Sysadmin	8 AM
Review uptime/performance	Daily	5 min	Sysadmin	9 AM
Database optimization	Weekly	15 min	DBA	Sun 2 AM
WordPress core updates	Weekly*	30 min	Sysadmin	Scheduled

Task	Frequency	Duration	Owner	Window
Plugin/theme updates	Weekly*	45 min	Sysadmin	Scheduled
Backup verification	Weekly	20 min	Sysadmin	Mon 6 AM
Security patch review	Weekly	30 min	Security	Any time
PHP version check	Monthly	1 hour	Sysadmin	Scheduled
Log rotation check	Monthly	10 min	Sysadmin	1st Sun
Firewall rule audit	Monthly	20 min	Sysadmin	1st Sun
Full backup check	Monthly	1 hour	DBA	Last Sun

* *Update as available; prioritize security updates immediately*

1.7.2 6.2 WordPress Core Updates

Time Estimate: 30-45 minutes

Prerequisites: - Database backup created - No deploys in progress - Monitor available for next 30 minutes

Steps:

1. **Check Current Version**

```
wp core version
wp core check-update
```

2. **Backup Database**

```
wp db export backup/pre-core-update-$(date +%Y%m%d-%H%M%S).sql
```

3. **Update WordPress Core**

```
# Check for updates
wp core update --dry-run

# Update WordPress
wp core update

# Update database schema if needed
wp core update-db
```

4. **Verify Update**

```
# Confirm new version
wp core version
```

```
# Check for errors in logs
tail -20 /var/log/php-errors.log
wp debug log tail --lines=20
```

5. **Clear Caches**

```
wp cache flush
wp w3-total-cache flush all
```

6. **Test Site**

   - Verify homepage loads: `curl -I https://[CUSTOMIZE: example.com]`
   - Test admin login
   - Check dashboard for warnings
   - Test critical user flows

> **WARNING:** Minor updates (6.4.1 to 6.4.2) are generally safe. Major updates (6.3 to 6.4) should be tested on staging first.

1.7.3 6.3 Plugin and Theme Updates

Time Estimate: 45-90 minutes depending on number of updates

Prerequisites: - All updates tested on staging - Database backed up - Monitor available during and after

Steps:

1. **List Available Updates**

```
wp plugin list --format=table --field=name,status,update
wp theme list --format=table --field=name,status,update
```

2. **Backup Database**

```
wp db export backup/pre-plugin-update-$(date +%Y%m%d-%H%M%S).sql
```

3. **Update Plugins (One by One)**

```
# For each plugin needing update:
wp plugin update plugin-name

# Verify no errors
tail -10 /var/log/php-errors.log

# Run a quick health check
wp health-check run
```

4. **Update Themes**

```
# If using child theme, update parent only
wp theme update parent-theme-name
```

5. **Verify Plugins are Active**

```
wp plugin status

# If a plugin deactivated due to error, troubleshoot:
wp plugin activate plugin-name --debug
```

6. **Post-Update Checks**

```
# Verify WordPress core data
wp db check

# Clear caches
wp cache flush

# Test critical pages
curl -s https://[CUSTOMIZE: example.com] | grep "<title>"
```

> **NOTE:** Keep a log of updates in your change management system. Document any issues encountered.

1.7.4 6.4 Database Optimization

Time Estimate: 15-30 minutes

Optimize Database (Non-Destructive):

```
# Check database size
wp db query "SELECT ROUND(SUM(data_length + index_length) / 1024 / 1024, 2) AS 'Database
↪  Size (MB)' FROM information_schema.tables WHERE table_schema = '[CUSTOMIZE:
↪  wordpress_db]';"

# Optimize all tables
wp db optimize

# Check for database errors
wp db check --repair

# View database statistics
wp db query "SHOW TABLE STATUS;" --format=table
```

Remove Old Revisions:

```
# Check revision count
wp post-meta list --meta-key='_wp_old_slug' --format=count
```

```
# Delete all revisions (keeps latest 5)
wp shell
< delete all post revisions keeping only last 5
> $results = $wpdb->get_results("SELECT post_id FROM {$wpdb->postmeta} WHERE meta_key =
 ↪  '_edit_lock'");
> # Then manually delete old revisions...
```

Or add to `wp-config.php` to limit future revisions:

```
define('WP_POST_REVISIONS', 5);
```

Clean Up Spam Comments:

```
# List spam comments
wp comment list --status=spam --format=table
 ↪  --field=comment_ID,comment_author,comment_date

# Delete spam comments
wp comment delete --force --status=spam
```

Remove Unused Transients:

```
# Check for expired transients
wp transient list --expired

# Delete all expired transients
wp transient delete --expired --all
```

1.7.5 6.5 PHP Version Upgrade

Time Estimate: 1-2 hours including testing

> **WARNING:** Major PHP version changes can break plugins/themes. Always test on staging first.

Prerequisites: - All plugins compatible with new PHP version (check WordPress.org) - All themes compatible - Staging environment updated first - Database backed up - 2-hour maintenance window available

Steps:

1. **Check Current PHP Version**

   ```
   php -v
   wp core version
   wp plugin list --field=name | while read plugin; do echo "=== $plugin ==="; wp
    ↪  plugin install $plugin --activate 2>&1 | grep -i "php\|version" || true; done
   ```

2. **Update PHP on Staging**

```
# Update PHP (example for Ubuntu/Debian)
sudo apt-get update
sudo apt-get install -y php8.2 php8.2-fpm php8.2-mysql php8.2-gd php8.2-xml
↪ php8.2-curl php8.2-mbstring php8.2-json

# Switch to new PHP version
sudo update-alternatives --set php /usr/bin/php8.2
```

3. **Update PHP-FPM Pool Configuration**

```
# Edit pool configuration
sudo nano /etc/php/8.2/fpm/pool.d/www.conf

# Restart PHP-FPM
sudo systemctl restart php8.2-fpm
```

4. **Test on Staging**

```
# Verify site works
curl -I https://[CUSTOMIZE: staging.example.com]

# Check for PHP errors
tail -30 /var/log/php8.2-fpm.log

# Run automated tests
npm run test
```

5. **Update Nginx Configuration** (if needed)

```
# In /etc/nginx/sites-available/default
upstream php {
    server unix:/run/php/php8.2-fpm.sock;  # Update version
}
```

6. **Deploy to Production**

```
# During maintenance window
sudo apt-get install -y php8.2-fpm php8.2-mysql php8.2-gd php8.2-xml php8.2-curl
↪ php8.2-mbstring php8.2-json

# Update Nginx
sudo nginx -t
sudo systemctl reload nginx

# Restart PHP-FPM
sudo systemctl restart php8.2-fpm

# Verify
curl -I https://[CUSTOMIZE: example.com]
```

7. **Monitor for Issues**
    - Watch error logs for 30 minutes
    - Test critical functionality
    - Verify form submissions work
    - Check database connections

1.7.6 6.6 WordPress Cron (WP-Cron) Management

What is WP-Cron:

WP-Cron uses WordPress hooks to trigger background tasks (email notifications, scheduled posts, cache cleaning, etc.) when site traffic happens. It's not a true system cron.

> **NOTE:** If your site has low traffic, WP-Cron tasks may not run on schedule. Consider using system cron instead.

Check WP-Cron Status:

```
# List all scheduled cron events
wp cron event list

# Check if WP-Cron is functioning
wp cron test

# Run all pending cron tasks manually
wp cron event run --all
```

Disable WP-Cron (Recommended):

Add to `wp-config.php`:

```
define('DISABLE_WP_CRON', true);
```

Use System Cron Instead:

Add to system crontab:

```
# Run WordPress cron every 5 minutes
*/5 * * * * curl -s https://[CUSTOMIZE: example.com]/wp-cron.php?doing_wp_cron >
 ↪  /dev/null 2>&1

# Or using WP-CLI (better)
*/5 * * * * wp cron event run --due-now --allow-root
```

1.7.7 6.7 Transactional Email Management

Email Delivery Configuration:

WordPress should send emails through an SMTP service, not PHP mail():

Using WP Mail SMTP Plugin:

```
# Install and activate
wp plugin install wp-mail-smtp --activate

# Configure SMTP via WP-CLI
wp option update mailer '{"mailer":"smtp"}'
wp option update mailer_host '[CUSTOMIZE: smtp.sendgrid.net]'
wp option update mailer_port '587'
wp option update mailer_encryption 'tls'
wp option update mailer_username '[CUSTOMIZE: apikey]'
# Note: Password should be stored securely, not in option
```

Or Configure in wp-config.php:

```
// Use Sendgrid or Postmark SMTP
define('SMTP_HOST', '[CUSTOMIZE: smtp.sendgrid.net]');
define('SMTP_PORT', 587);
define('SMTP_USER', '[CUSTOMIZE: apikey]');
define('SMTP_PASSWORD', '[CUSTOMIZE: sg.xxx...]');
define('SMTP_FROM', 'noreply@[CUSTOMIZE: example.com]');

// Hook to override WordPress mail function
add_filter('wp_mail', 'use_smtp_mail');
function use_smtp_mail($atts) {
    // Implementation uses defined constants above
    return $atts;
}
```

Test Email Delivery:

```
# Send test email
wp eval-file - <<'PHP'
wp_mail('[CUSTOMIZE: admin@example.com]', 'WordPress Email Test', 'This is a test email
↪  from WordPress.');
PHP

# Check for errors
tail -20 /var/log/php-errors.log
```

1.7.8 6.8 Log Rotation

Configure Log Rotation (`/etc/logrotate.d/wordpress`):

```
/var/log/nginx/*.log {
    daily
    missingok
```

```
    rotate 14
    compress
    delaycompress
    notifempty
    create 0640 www-data adm
    sharedscripts
    postrotate
        if [ -f /var/run/nginx.pid ]; then
            kill -USR1 `cat /var/run/nginx.pid`
        fi
    endscript
}

/home/wordpress/logs/*.log {
    daily
    missingok
    rotate 30
    compress
    delaycompress
    notifempty
    create 0640 www-data www-data
}

/var/log/php*.log {
    daily
    missingok
    rotate 14
    compress
    delaycompress
    notifempty
    sharedscripts
    postrotate
        /usr/lib/php/php-fpm-checkconf > /dev/null 2>&1 || true
        if [ $? -eq 0 ]; then
            systemctl reload php8.1-fpm > /dev/null 2>&1 || true
        fi
    endscript
}

/var/log/mysql/error.log {
    daily
    missingok
    rotate 14
    compress
    delaycompress
    notifempty
    create 640 mysql mysql
```

```
}
```

Test Log Rotation:

```
# Test logrotate configuration
sudo logrotate -d /etc/logrotate.d/wordpress

# Force immediate rotation (testing only)
sudo logrotate -f /etc/logrotate.d/wordpress

# Verify rotation worked
ls -lah /var/log/nginx/*.log*
```

1.7.9 6.9 Monitoring and Alerting

Key Metrics to Monitor:

Metric	Threshold	Check Frequency
Uptime	99.9%	Every 5 minutes
Response Time	< 2 seconds	Every 60 seconds
CPU Usage	< 80%	Every 60 seconds
Memory Usage	< 90%	Every 60 seconds
Disk Space	> 20% free	Every 60 seconds
Error Rate	< 1%	Every 60 seconds

Using Prometheus + Grafana (Optional):

```
# Install node_exporter for server metrics
wget
 ↪  https://github.com/prometheus/node_exporter/releases/download/v1.6.0/node_exporter-
 ↪  1.6.0.linux-amd64.tar.gz
tar xvfz node_exporter-1.6.0.linux-amd64.tar.gz
sudo mv node_exporter-1.6.0.linux-amd64/node_exporter /usr/local/bin/

# Create systemd service
sudo tee /etc/systemd/system/node_exporter.service > /dev/null <<EOF
[Unit]
Description=Node Exporter
After=network.target

[Service]
User=prometheus
ExecStart=/usr/local/bin/node_exporter
```

```
[Install]
WantedBy=multi-user.target
EOF

sudo systemctl daemon-reload
sudo systemctl enable node_exporter
sudo systemctl start node_exporter
```

Simple Monitoring Script:

```
#!/bin/bash
# Check WordPress health every 5 minutes

SITE_URL="https://[CUSTOMIZE: example.com]"
RESPONSE=$(curl -s -w "%{http_code}" -o /dev/null $SITE_URL)

if [ "$RESPONSE" != "200" ]; then
    echo "ALERT: Site returned $RESPONSE at $(date)" | mail -s "WordPress Site Down"
     ↪  root@example.com
    # Attempt to restart PHP-FPM
    systemctl restart php8.1-fpm
fi

# Check database
if ! wp db check > /dev/null 2>&1; then
    echo "ALERT: Database check failed at $(date)" | mail -s "WordPress DB Error"
     ↪  root@example.com
fi

# Check disk space
DISK_USAGE=$(df /home/wordpress | awk 'NR==2 {print $5}' | sed 's/%//')
if [ "$DISK_USAGE" -gt 90 ]; then
    echo "ALERT: Disk usage at ${DISK_USAGE}% at $(date)" | mail -s "WordPress Disk
     ↪  Full" root@example.com
fi
```

Add to crontab:

```
*/5 * * * * /usr/local/bin/wordpress-health-check.sh
```

1.8 Section 7: Backup Procedures

1.8.1 7.1 Backup Strategy

Component	Frequency	Method	Retention	Storage
Database	Every 6 hours	`wp db export` + mysqldump	90 days	[CUSTOMIZE: S3 / Backblaze / NAS]
WordPress Files	Daily	rsync/tar	30 days	[CUSTOMIZE: S3 / Backblaze / NAS]
Uploads Directory	Daily	rsync	30 days	[CUSTOMIZE: S3 / Backblaze / NAS]
Configuration Files	Weekly	git + encrypted backup	1 year	[CUSTOMIZE: Git repo + vault]
Full Site Snapshot	Weekly	Volume snapshot	8 weeks	[CUSTOMIZE: VM snapshot / AWS EBS]

> **WARNING:** Test all backups quarterly. A backup that has never been restored is not a backup.

1.8.2 7.2 Automated Backup Script

Create `/usr/local/bin/wordpress-backup.sh`:

```
#!/bin/bash
set -e

BACKUP_DIR="/home/wordpress/backup"
SITE_URL="[CUSTOMIZE: example.com]"
DB_NAME="[CUSTOMIZE: wordpress]"
DB_USER="[CUSTOMIZE: wp_user]"
RETENTION_DAYS=90
TIMESTAMP=$(date +%Y%m%d_%H%M%S)
LOG_FILE="/var/log/wordpress-backup.log"

# Create backup directory if it doesn't exist
mkdir -p "$BACKUP_DIR"

echo "[$(date)] Starting WordPress backup..." >> "$LOG_FILE"

# 1. Database backup
echo "[$(date)] Backing up database..." >> "$LOG_FILE"
```

```
wp db export "$BACKUP_DIR/db_${TIMESTAMP}.sql" --single-transaction >> "$LOG_FILE" 2>&1
gzip "$BACKUP_DIR/db_${TIMESTAMP}.sql"

# 2. WordPress files backup (excluding uploads and cache)
echo "[$(date)] Backing up WordPress files..." >> "$LOG_FILE"
tar --exclude='wp-content/uploads' --exclude='wp-content/cache' \
    --exclude='.git' --exclude='node_modules' \
    -czf "$BACKUP_DIR/wordpress_${TIMESTAMP}.tar.gz" \
    /home/wordpress/public_html >> "$LOG_FILE" 2>&1

# 3. Uploads directory backup
echo "[$(date)] Backing up uploads..." >> "$LOG_FILE"
tar -czf "$BACKUP_DIR/uploads_${TIMESTAMP}.tar.gz" \
    /home/wordpress/public_html/wp-content/uploads >> "$LOG_FILE" 2>&1

# 4. Upload to remote storage (example using AWS S3)
echo "[$(date)] Uploading to S3..." >> "$LOG_FILE"
aws s3 sync "$BACKUP_DIR" "s3://[CUSTOMIZE: backup-bucket]/${SITE_URL}/" \
    --exclude="*" --include="*_${TIMESTAMP}.tar.gz" --include="*_${TIMESTAMP}.sql.gz"
    ↪  >> "$LOG_FILE" 2>&1

# 5. Cleanup old local backups
echo "[$(date)] Cleaning up old backups..." >> "$LOG_FILE"
find "$BACKUP_DIR" -type f -mtime +$RETENTION_DAYS -delete

# 6. Verify backup integrity
echo "[$(date)] Verifying backup..." >> "$LOG_FILE"
for file in "$BACKUP_DIR"/*_${TIMESTAMP}.*; do
    if [ -f "$file" ]; then
        SIZE=$(du -h "$file" | cut -f1)
        echo "[$(date)] Backup file: $(basename $file) (Size: $SIZE)" >> "$LOG_FILE"
    fi
done

echo "[$(date)] Backup completed successfully" >> "$LOG_FILE"
```

Make Script Executable:

```
sudo chmod +x /usr/local/bin/wordpress-backup.sh
sudo chown root:root /usr/local/bin/wordpress-backup.sh
```

Schedule with Cron:

```
# Run backup every 6 hours
0 */6 * * * /usr/local/bin/wordpress-backup.sh

# Run weekly full snapshot at 2 AM Sunday
0 2 * * 0 /usr/local/bin/wordpress-backup.sh
```

1.8.3 7.3 Backup Verification

Monthly Backup Test:

Time Estimate: 30-45 minutes

1. **List Available Backups**

```
ls -lh /home/wordpress/backup/ | tail -20

# Or check S3
aws s3 ls s3://[CUSTOMIZE: backup-bucket]/[CUSTOMIZE: example.com]/
↪  --human-readable
```

2. **Verify Backup Integrity**

```
# Test database export
gunzip -t /home/wordpress/backup/db_*.sql.gz

# Verify tar archives
tar -tzf /home/wordpress/backup/wordpress_*.tar.gz | head -20
tar -tzf /home/wordpress/backup/uploads_*.tar.gz | head -20
```

3. **Perform Test Restore (On Staging)**

```
# Extract database backup
zcat /home/wordpress/backup/db_TIMESTAMP.sql.gz > /tmp/test-restore.sql

# Import to test database
wp db import /tmp/test-restore.sql --allow-root

# Verify tables
wp db table list --all-tables --format=table
```

4. **Test File Restore**

```
# Extract to temporary location
mkdir -p /tmp/restore-test
tar -xzf /home/wordpress/backup/wordpress_TIMESTAMP.tar.gz -C /tmp/restore-test

# Verify key files exist
ls -la /tmp/restore-test/home/wordpress/public_html/wp-config.php
ls -la /tmp/restore-test/home/wordpress/public_html/index.php
```

5. **Document Results**

```
Backup Verification Report - [DATE]

▫ Database backup: [SIZE], verified
▫ WordPress files backup: [SIZE], verified
```

```
    ☐ Uploads backup: [SIZE], verified
    ☐ Test restore successful
    ☐ All files present and readable

    Next verification: [DATE 30 days from now]
```

1.9 Section 8: Deployment Procedures

1.9.1 8.1 Code Deployment

See Section 4.2 for complete deployment workflow.

Quick Reference:

```
# Pull code and deploy
cd /home/wordpress/public_html
git fetch origin
git checkout release/version-X.Y.Z
composer install --no-dev

# Run WordPress updates
wp plugin update --all
wp theme update --all
wp core update-db

# Verify and test
wp core is-installed && echo "OK"
curl -I https://[CUSTOMIZE: example.com]
```

1.9.2 8.2 Database Migration

Time Estimate: 30-60 minutes

> **WARNING:** Always backup before any database migration. This is high-risk.

Prerequisites: - Database backup created and verified - Change window scheduled - Rollback plan documented - All users notified

Steps:

1. **Export Database from Source**

   ```
   # From old server
   wp db export backup/migration-source-$(date +%Y%m%d).sql --single-transaction
   ```

```
# Or using mysqldump
mysqldump -u [CUSTOMIZE: user] -p --single-transaction --quick \
  [CUSTOMIZE: database_name] > backup/migration-source-$(date +%Y%m%d).sql
```

2. **Transfer Database**

```
# Secure copy to new server
scp backup/migration-source-*.sql [CUSTOMIZE: user@new-server]:/tmp/
```

3. **Prepare New Database**

```
# On new server - create empty database
wp db create

# Or manually
mysql -u root -p
> CREATE DATABASE [CUSTOMIZE: new_database_name];
> CREATE USER '[CUSTOMIZE: wp_user]'@'localhost' IDENTIFIED BY '[password]';
> GRANT ALL PRIVILEGES ON [CUSTOMIZE: new_database_name].* TO '[CUSTOMIZE:
 ↪  wp_user]'@'localhost';
> FLUSH PRIVILEGES;
```

4. **Import Database**

```
# On new server
wp db import /tmp/migration-source-*.sql

# Or using mysql command
mysql -u [CUSTOMIZE: user] -p [CUSTOMIZE: database] < /tmp/migration-source-*.sql
```

5. **Update WordPress URLs** (if hostname changed)

```
# Search and replace old domain with new
wp search-replace "https://old.example.com" "https://new.example.com" --all-tables

# Update WordPress options
wp option update home "https://[CUSTOMIZE: new.example.com]"
wp option update siteurl "https://[CUSTOMIZE: new.example.com]"
```

6. **Verify Database**

```
# Check integrity
wp db check

# Verify tables count
wp db table list --all-tables --format=table

# Check for errors
wp db query "SHOW ENGINE INNODB STATUS\G" | head -50
```

7. **Clear Caches**

```
wp cache flush
wp w3-total-cache flush all
wp redis flush-db
```

8. **Monitor for Issues**
   - Watch error logs
   - Test critical pages
   - Verify user login works
   - Check media URLs point to new location

1.9.3 8.3 Rollback Procedure

Time Estimate: 15-30 minutes

> **CRITICAL:** Test rollback procedure quarterly.

Scenario: Deployment introduced critical errors and site is down or broken.

Steps:

1. **Identify Current State**

```
wp core version
git log --oneline -5
git status
```

2. **Revert Code to Previous Release**

```
# If using git
git checkout previous-release-tag
# OR
git revert HEAD --no-edit  # Revert last commit

# OR manually restore from backup
tar -xzf /home/wordpress/backup/wordpress_TIMESTAMP.tar.gz -C /
```

3. **Revert Database if Needed**

```
# Restore from pre-deployment backup
wp db import backup/pre-deploy-TIMESTAMP.sql

# Or if using database transactions
wp db query "ROLLBACK;"
```

4. **Verify Rollback**

```
wp core version
curl -I https://[CUSTOMIZE: example.com]
wp debug log tail --lines=30
```

5. **Investigate Root Cause**

```
# Review error logs
tail -100 /var/log/php-errors.log

# Check plugin compatibility
wp plugin list --status=inactive --format=table

# Review recent changes
git diff [old-version]...[new-version] | head -100
```

6. **Document Incident**

   - What went wrong
   - When discovered
   - Rollback time
   - Root cause
   - Prevention measures

1.10 Section 9: Content Operations

1.10.1 9.1 Media Management

Upload Folder Structure:

WordPress organizes uploads by year and month:

```
wp-content/uploads/
├── 2026/
│   ├── 01/
│   │   ├── image.jpg
│   │   ├── image-150x150.jpg  (thumbnail)
│   │   └── image-300x300.jpg  (medium)
│   └── 02/
│       └── ...
└── cache/  (plugin caching)
```

Image Optimization with WebP:

> **CAVEAT:** Not all browsers support WebP. Always maintain JPEG/PNG fallbacks.

```
# Install image optimization plugin
wp plugin install wp-smushit --activate
```

```
# Or configure in nginx for automatic WebP delivery
location ~* ^.+\.(jpg|jpeg|gif|png)$ {
    # Serve WebP if browser accepts it and file exists
    if ($http_accept ~* "webp") {
        rewrite ^(.*)$ $1.webp break;
    }
    expires 30d;
    add_header Cache-Control "public, immutable";
}
```

Monitor Upload Folder Size:

```
# Check upload directory size
du -sh /home/wordpress/public_html/wp-content/uploads/

# Find large files
find /home/wordpress/public_html/wp-content/uploads/ -type f -size +10M -exec ls -lh {}
 ↪  \;

# Clean up orphaned images (use caution)
wp media regenerate --yes  # Regenerate thumbnails
```

Offload Media to CDN (Optional):

```
# Install offload plugin
wp plugin install wordpress-unlimited-amazon-s3-media-library --activate

# Configure S3 bucket
wp option update as3cf_settings '{
  "bucket": "[CUSTOMIZE: my-bucket]",
  "region": "[CUSTOMIZE: us-east-1]"
}'
```

1.10.2 9.2 Publishing Workflow

Content Calendar (Monthly Table Example):

Date	Post Title	Author	Status	Notes
Feb 1	Spring Campaign Launch	Jane	Draft	Needs final review
Feb 5	Product Update	Bob	Scheduled	Publishing at 9 AM
Feb 10	Blog Series Pt. 1	Alice	Published	2,500 views

Create and Schedule Posts:

```
# Create draft post
wp post create --post_type=post --post_title='New Post' \
  --post_content='Post content here' --post_status=draft

# Schedule post for future publishing
wp post create --post_type=post --post_title='Scheduled Post' \
  --post_content='Content' --post_status=scheduled \
  --post_date='2026-02-20 09:00:00'

# List scheduled posts
wp post list --post_status=scheduled --format=table
```

Review Before Publishing:

1. Content review for accuracy and tone
2. Check for broken links: `wp link list --format=table`
3. Review SEO metadata (if using Yoast)
4. Verify images display correctly
5. Test on mobile device
6. Check URL/permalink is correct

Publish Post:

```
# Transition draft to published
wp post update [POST_ID] --post_status=publish

# Verify publication
wp post get [POST_ID] --format=table
```

1.10.3 9.3 Taxonomy and Permalink Management

Categories and Tags:

```
# List all categories
wp term list category --format=table

# Create new category
wp term create category "New Category" --description="Category description"

# Update category
wp term update category [TERM_ID] --name="Updated Name"

# Delete category (reassign posts)
wp term delete category [TERM_ID] --default=1  # Assign to 'Uncategorized'
```

Permalink Structure:

Current structure: [CUSTOMIZE: /%year%/%monthnum%/%postname%/]

```
# View current permalink structure
wp option get permalink_structure

# Change permalink structure (takes effect immediately)
wp rewrite structure '/%year%/%monthnum%/%postname%/'

# Flush rewrite rules
wp rewrite flush

# Regenerate 301 redirects for old URLs (if changed)
wp plugin install redirection --activate
wp redirect rule create 'old-url' 'new-url' --type=301
```

> **WARNING:** Changing permalink structure requires 301 redirects from old URLs. Update internal links and notify users.

Custom Post Types:

```
# List custom post types
wp post-type list --format=table

# Create custom post type programmatically
wp plugin install [custom-plugin] --activate

# Register in functions.php
add_action('init', function() {
    register_post_type('portfolio', array(
        'public' => true,
        'label' => 'Portfolio',
        'supports' => array('title', 'editor', 'thumbnail'),
    ));
});
```

1.10.4 9.4 Search and Indexing

WordPress Search Optimization:

```
# Verify WP search is working
wp search-term "keyword" --post_type=post

# Rebuild search index (if using search plugin)
wp plugin activate wp-search-suggest
wp search-suggest rebuild-index

# Check for posts with no content (unfixed)
wp db query "SELECT ID, post_title FROM wp_posts WHERE post_content = '' AND post_type =
↪  'post';"
```

Search Engine Indexing:

```
# Check if site is visible to search engines
wp option get blog_public

# Enable search engine visibility
wp option update blog_public 1

# Submit sitemap to Google Search Console
wp seo-sitemap generate

# List registered sitemaps
wp sitemap list
```

WP-CLI Search and Replace:

```
# Search for content
wp search-replace "old-text" "new-text" --dry-run  # Preview changes

# Execute search-replace
wp search-replace "old-text" "new-text"

# Search and replace in specific posts
wp search-replace "old" "new" --post_id=123,456,789
```

1.11 Section 10: Incident Response

1.11.1 10.1 Severity Classification

Severity	Impact	Response Time	Example
Critical (P1)	Site completely down or major data loss	15 minutes	Database corruption, ransomware, all users locked out
High (P2)	Core functionality broken, partial outage	30 minutes	Plugin causing 500 errors, login broken, media not loading
Medium (P3)	Non-critical feature broken or degraded	4 hours	Search not working, theme display issues

Severity	Impact	Response Time	Example
Low (P4)	Minor issues, cosmetic or UX	1-2 business days	Typo, formatting issue, slow report generation

1.11.2 10.2 Site Down / 500 Error Triage

Time Estimate: 5-10 minutes to identify root cause

Symptoms: - Browser shows 500 error or blank page - Site returns HTTP 500 or 502/503 - `curl` returns error

Immediate Actions:

1. **Verify Site is Actually Down**

```
# Test from multiple locations
curl -I https://[CUSTOMIZE: example.com]
curl -I https://[CUSTOMIZE: example.com]/wp-admin/

# Check HTTP status code
curl -s -w "%{http_code}\n" -o /dev/null https://[CUSTOMIZE: example.com]
```

2. **Check Server Status**

```
# Check if services are running
systemctl status nginx
systemctl status php8.1-fpm
systemctl status mysql

# Check server load
top -bn1 | head -20
free -h
df -h /home/wordpress

# Check for disk space
df -h | grep -E "^/dev|^Filesystem"
```

3. **Check Error Logs**

```
# PHP errors
tail -30 /var/log/php-errors.log | grep -A5 -B5 error

# Nginx errors
tail -30 /var/log/nginx/error.log
```

```
# MySQL errors
tail -30 /var/log/mysql/error.log

# WordPress debug log
wp debug log tail --lines=30
```

4. **Disable Recently Activated Plugins**

```
# List recently activated plugins
wp plugin list --status=active | grep -E "activation|installed"

# Deactivate all plugins (one by one, testing after each)
wp plugin deactivate --all

# Test site
curl -I https://[CUSTOMIZE: example.com]

# If fixed, reactivate plugins one by one to find culprit
wp plugin activate plugin-name-1
curl -I https://[CUSTOMIZE: example.com]
# Continue...
```

5. **Check Database Connection**

```
wp db check

# If error:
wp db repair
wp db optimize
```

6. **Check Memory Limits**

```
# Current PHP memory limit
php -r "echo ini_get('memory_limit');"

# If low, increase (in wp-config.php)
define('WP_MEMORY_LIMIT', '256M');
define('WP_MAX_MEMORY_LIMIT', '512M');
```

7. **Restart Services if Needed**

```
# Restart in this order
sudo systemctl restart mysql
sleep 5
sudo systemctl restart php8.1-fpm
sleep 5
sudo systemctl restart nginx

# Test
curl -I https://[CUSTOMIZE: example.com]
```

If Still Down: - Check cloud provider status page for outages - Restore from recent backup (Section 11.2) - Escalate to senior engineer

1.11.3 10.3 Security Breach Response

Time Estimate: 1-4 hours depending on scope

> **CRITICAL:** If breach is suspected, act immediately. Data loss and reputation damage increase with every minute of delay.

Symptoms: - Unexpected files appear in directory - Hacker message on site - Users report password changes they didn't make - Unexpected admin accounts - Site redirects to malicious site - Malware detected by security scanner

Immediate Actions (First 15 Minutes):

1. **Notify Security Team Immediately**

```
Severity: CRITICAL
Issue: [Description of attack]
Time Detected: [Time]
Affected User Data: [If known]
Escalation: [Your contact details]
```

2. **Isolate the Site**

```
# Take site offline to prevent further data exfiltration
# Option 1: Redirect to maintenance page
wp maintenance-mode activate

# Option 2: Block all traffic except admins
# Add to .htaccess or nginx config:
# deny all;
# allow [CUSTOMIZE: your-ip];
```

3. **Identify Breach Scope**

```
# Check for suspicious accounts
wp user list --format=table
wp user list --role=administrator --format=table

# Look for unexpected users (compare to known list)
# Delete if suspicious:
wp user delete [USER_ID] --reassign=[ADMIN_ID]
```

4. **Check for Web Shells**

```
# Find recently modified PHP files
find /home/wordpress -name "*.php" -type f -mtime -7 -ls

# Look for suspicious files
find /home/wordpress -name "shell.php" -o -name "admin.php" -o -name "tmp*.php"

# Remove if found (back up first for forensics)
cp /home/wordpress/public_html/suspicious-file.php \
  /root/forensics/suspicious-file.php.bak
rm /home/wordpress/public_html/suspicious-file.php
```

5. **Force Password Resets**

```
# Reset all admin passwords
wp user list --role=administrator --format=table --field=user_login | while read
 ↪  user; do
  NEW_PASS=$(openssl rand -base64 16)
  wp user update $user --prompt=user_pass
  echo "Reset $user to temporary password"
done

# Force users to reset passwords on next login
wp option update force_password_reset true
```

6. **Scan for Malware**

```
# Using Wordfence or other security plugin
wp plugin install wordfence --activate
wp wordfence scan full

# Or use AIDE for file integrity
aide --check > /tmp/aide-report.txt
```

7. **Backup Everything for Forensics**

```
# Before cleaning, backup entire site for investigation
tar -czf /root/forensics/breach-evidence-$(date +%Y%m%d-%H%M%S).tar.gz \
  /home/wordpress/public_html

# Export database
wp db export /root/forensics/breach-evidence-db-$(date +%Y%m%d-%H%M%S).sql
```

8. **Clean the Site**

```
# Remove malware files
wp plugin delete infected-plugin-name --deactivate

# Update all plugins, themes, WordPress core
wp core update
```

```
wp plugin update --all
wp theme update --all

# Change all passwords and API keys
# (Regenerate WordPress security keys in wp-config.php)
```

9. **Monitor Closely**

```
# Watch for re-compromise
tail -f /var/log/nginx/access.log | grep -i "shell\|admin\|eval"
watch -n 5 'ps aux | grep php'

# Check for suspicious cron jobs
crontab -l
wp cron event list
```

Post-Incident: - Contact affected users - Monitor security vendor alerts - File incident report - Update security measures - Schedule security audit

1.11.4 10.4 Escalation Path

Contact Escalation Order:

1. **On-Call SysAdmin** (respond within 30 min)
   - Phone: [CUSTOMIZE]
   - Email: [CUSTOMIZE]
   - Slack: [CUSTOMIZE]
2. **On-Call Database Admin** (respond within 60 min)
   - Phone: [CUSTOMIZE]
   - Email: [CUSTOMIZE]
   - Slack: [CUSTOMIZE]
3. **Security Officer** (for security incidents)
   - Phone: [CUSTOMIZE]
   - Email: [CUSTOMIZE]
   - Slack: [CUSTOMIZE]
4. **Manager/Director** (for critical incidents)
   - Phone: [CUSTOMIZE]
   - Email: [CUSTOMIZE]
   - Slack: [CUSTOMIZE]
5. **Incident Commander** (if declared critical)

- Phone: [CUSTOMIZE]
- Email: [CUSTOMIZE]
- Slack: [CUSTOMIZE]

1.11.5 10.5 Performance Degradation

Time Estimate: 15-30 minutes to identify cause

Symptoms: - Page load > 2 seconds - High CPU usage (>80%) - High memory usage (>90%) - Database slow queries - Users report site sluggish

Diagnostic Steps:

1. **Check Current Performance**

```
# Load average
uptime

# CPU usage
top -bn1 | head -15

# Memory
free -h

# Response time
time curl -s https://[CUSTOMIZE: example.com] > /dev/null
```

2. **Identify Bottleneck**

```
# MySQL slow query log
tail -50 /var/log/mysql/slow.log

# PHP profiling
wp plugin install query-monitor --activate
# View profiling at https://example.com/?qm-uuid=...

# Nginx access log analysis
tail -1000 /var/log/nginx/access.log | awk '{print $NF}' | sort | uniq -c | sort
↪  -rn | head
```

3. **Check Common Causes**

```
# Full disk
df -h /home/wordpress

# Database size
wp db query "SELECT ROUND(SUM(data_length + index_length) / 1024 / 1024, 2) AS
↪  'Size (MB)' FROM information_schema.tables WHERE table_schema = '[CUSTOMIZE:
↪  wordpress_db]';"
```

```
# Large posts/revisions
wp post list --orderby=post_date --order=DESC --posts_per_page=100 --format=table

# Runaway cron jobs
wp cron event list
ps aux | grep "wp cron"
```

4. **Optimize Database**

```
# Delete old revisions
wp db query "DELETE FROM wp_posts WHERE post_type = 'revision';"

# Optimize tables
wp db optimize

# Delete expired transients
wp transient delete --expired --all
```

5. **Clear Caches**

```
wp cache flush
wp w3-total-cache flush all
wp redis flush-db

# Clear browser cache headers
curl -I https://[CUSTOMIZE: example.com] | grep Cache-Control
```

6. **Monitor Results**

```
# Wait 5 minutes and test again
time curl -s https://[CUSTOMIZE: example.com] > /dev/null
uptime
free -h
```

1.11.6 10.6 Post-Incident Review

Complete within 24 hours of incident resolution

Documentation:

```
# Incident Report - [DATE]

## Summary
[1-2 sentence overview]

## Severity
[Critical/High/Medium/Low]
```

```
## Timeline
- HH:MM - Issue first detected
- HH:MM - Root cause identified
- HH:MM - Incident resolved
- HH:MM - Post-incident review started

## Root Cause
[Detailed explanation of what happened]

## Impact
- Duration: [X minutes]
- Users affected: [number or percentage]
- Data loss: [Yes/No and details]
- Revenue impact: [estimate if applicable]

## Resolution
[Steps taken to resolve]

## Prevention
1. [Action to prevent recurrence]
2. [Implement monitoring]
3. [Update runbook]

## Action Items
- [ ] [Task] - Owner: [Name] - Due: [Date]
- [ ] [Task] - Owner: [Name] - Due: [Date]

## Participants
- [Name] - [Role]
- [Name] - [Role]

## Approval
Reviewed by: [Name]
Date: [Date]
```

1.12 Section 11: Disaster Recovery

1.12.1 11.1 Recovery Objectives

Metric	Target	Notes
RTO (Recovery Time Objective)	1 hour	Maximum acceptable downtime
RPO (Recovery Point Objective)	6 hours	Maximum acceptable data loss
MTTR (Mean Time To Recovery)	30 minutes	Average time to recover
Backup Frequency	Every 6 hours	Database, 4x daily
Backup Retention	90 days	Monthly snapshots kept 1 year
Recovery Testing	Quarterly	Test recovery from backups

> **NOTE:** These targets assume proper backups and infrastructure in place. Regular testing is critical.

1.12.2 11.2 Full Site Restore from Backup

Time Estimate: 1-1.5 hours

When to Use: - Complete server failure or data corruption - Ransomware or major security breach - Accidental data deletion - Need to roll back to specific point in time

Prerequisites: - Backup verified and accessible - New server ready (or old server reformatted) - Database backup in hand - WordPress file backup in hand

Steps:

1. **Prepare Clean Server**

```
# If using new server, ensure LEMP stack is installed
# See Section 3.1 for architecture overview

# If reusing old server, clean it
sudo rm -rf /home/wordpress/public_html/*
sudo rm -rf /var/lib/mysql/[CUSTOMIZE: wordpress_db]/*
```

2. **Restore WordPress Files**

```
# Option 1: From local backup
tar -xzf /home/wordpress/backup/wordpress_TIMESTAMP.tar.gz -C /

# Option 2: From S3
aws s3 cp s3://[CUSTOMIZE: backup-bucket]/wordpress_TIMESTAMP.tar.gz - | tar -xz
↪  -C /
```

```
# Verify files exist
ls -la /home/wordpress/public_html/wp-config.php
```

3. **Restore Database**

```
# Create database if not exists
mysql -u root -p <<EOF
CREATE DATABASE [CUSTOMIZE: wordpress_db];
CREATE USER '[CUSTOMIZE: wp_user]'@'localhost' IDENTIFIED BY '[password]';
GRANT ALL PRIVILEGES ON [CUSTOMIZE: wordpress_db].* TO '[CUSTOMIZE:
↪ wp_user]'@'localhost';
FLUSH PRIVILEGES;
EOF

# Restore from backup
# Option 1: From local backup
wp db import /home/wordpress/backup/db_TIMESTAMP.sql

# Option 2: From S3
aws s3 cp s3://[CUSTOMIZE: backup-bucket]/db_TIMESTAMP.sql.gz - | gunzip | wp db
↪ import -
```

4. **Restore Permissions**

```
# Set correct ownership
sudo chown -R www-data:www-data /home/wordpress/public_html

# Set correct permissions (see Appendix A)
find /home/wordpress/public_html -type f -exec chmod 644 {} \;
find /home/wordpress/public_html -type d -exec chmod 755 {} \;
chmod 600 /home/wordpress/public_html/wp-config.php
```

5. **Restore Configuration Files**

```
# Verify wp-config.php has correct database credentials
grep "DB_NAME\|DB_USER\|DB_HOST" /home/wordpress/public_html/wp-config.php

# Update wp-config.php if needed
wp config set DB_NAME [CUSTOMIZE: wordpress_db]
wp config set DB_USER [CUSTOMIZE: wp_user]
```

6. **Verify Restoration**

```
# Check WordPress core
wp core is-installed && echo "WordPress installed"
wp core version

# Check database
wp db check
```

```
wp db table list --all-tables | head

# Check plugins
wp plugin list --status=active --format=table

# Check theme
wp theme list --status=active
```

7. **Update URLs** (if hostname changed)

```
# Update WordPress site URL
wp option update home "https://[CUSTOMIZE: example.com]"
wp option update siteurl "https://[CUSTOMIZE: example.com]"

# Search and replace old URLs
wp search-replace "https://old.example.com" "https://[CUSTOMIZE: example.com]"
↪  --all-tables
```

8. **Restore Services**

```
# Clear caches
wp cache flush
wp w3-total-cache flush all
wp redis flush-db

# Restart services
sudo systemctl restart nginx
sudo systemctl restart php8.1-fpm

# Test site
curl -I https://[CUSTOMIZE: example.com]
```

9. **Post-Restore**

   - Reset admin passwords
   - Check all users can login
   - Verify media loads correctly
   - Test form submissions
   - Monitor error logs for 30 minutes

1.12.3 11.3 Database-Only Recovery

Time Estimate: 20-30 minutes

When to Use: - Data corruption in database only - Accidental database edits/deletions - Database gone, but files are fine - Want to recover to specific point in time

Steps:

1. **Identify Target Backup**

```
# List available database backups
ls -lh /home/wordpress/backup/db_*.sql.gz

# Or from S3
aws s3 ls s3://[CUSTOMIZE: backup-bucket]/ | grep "db_"
```

2. **Backup Current Database** (in case needed for comparison)

```
wp db export /tmp/current-db-$(date +%Y%m%d-%H%M%S).sql
```

3. **Drop Current Database Tables**

```
# Drop all tables (careful!)
wp db reset --yes

# Or manually
mysql -u root -p [CUSTOMIZE: wordpress_db] <<EOF
DROP TABLE IF EXISTS wp_commentmeta;
DROP TABLE IF EXISTS wp_comments;
-- ... drop all tables
EOF
```

4. **Import Backup**

```
# From local backup
gunzip < /home/wordpress/backup/db_TIMESTAMP.sql.gz | wp db import -

# Or if backup is already uncompressed
wp db import /home/wordpress/backup/db_TIMESTAMP.sql
```

5. **Verify Restoration**

```
# Check database integrity
wp db check

# Count posts (compare to expected)
wp post list --post_type=all --format=count

# Verify users
wp user list --format=table
```

6. **Update and Clear Caches**

```
# If recovered to different time, update site metadata
wp option update last_update_check '0'

# Clear all caches
wp cache flush --global
```

```
wp w3-total-cache flush all
wp redis flush-db
```

7. **Monitor Site**
   - Check error logs
   - Test user login
   - Verify recent posts are present (or missing if intended)

1.12.4 11.4 Server Migration

Time Estimate: 2-4 hours

Scenario: Moving WordPress to a new server (new data center, cloud provider, hardware upgrade, etc.)

Prerequisites: - New server with LEMP stack installed and configured - DNS access for domain - SSH access to both servers - Current database backup - WordPress file backup

Steps:

1. **Prepare New Server**

```
# Verify all services running
systemctl status nginx
systemctl status php8.1-fpm
systemctl status mysql
systemctl status redis

# Create WordPress directory
sudo mkdir -p /home/wordpress/public_html
sudo chown www-data:www-data /home/wordpress/public_html
```

2. **Backup Current Site**

```
# Full backup before migration
/usr/local/bin/wordpress-backup.sh

# Verify backup integrity
ls -lh /home/wordpress/backup/*_$(date +%Y%m%d).*
```

3. **Restore to New Server**

```
# Follow Section 11.2 steps to restore files and database
```

4. **Update DNS**

```
# Update DNS A record to point to new server IP
# Verify DNS propagation
nslookup [CUSTOMIZE: example.com]
dig [CUSTOMIZE: example.com] +short
```

```
# Wait for TTL to expire (typically 1-24 hours)
```

5. **SSL Certificate**

```
# If using Let's Encrypt, certificate follows domain
# Verify certificate on new server
openssl s_client -connect [CUSTOMIZE: example.com]:443 -tls1_2

# If manual certificate, copy to new server
scp /etc/letsencrypt/live/[CUSTOMIZE: example.com]/* \
  root@new-server:/etc/letsencrypt/live/[CUSTOMIZE: example.com]/
```

6. **Test on New Server**

```
# Test via IP address (before DNS updates)
curl -H "Host: [CUSTOMIZE: example.com]" -I https://[new-ip-address]

# Once DNS updated, test normally
curl -I https://[CUSTOMIZE: example.com]
```

7. **Decommission Old Server**

```
# Do NOT immediately shut down old server
# Monitor new server for 24-48 hours first

# After verification period, backup old server data
tar -czf /backup/old-server-final-$(date +%Y%m%d).tar.gz /home/wordpress/

# Then safely shut down
sudo shutdown -h now
```

8. **Post-Migration Verification**

   - All users can access site
   - HTTPS certificate valid
   - All plugins active
   - Database working
   - Backups configured on new server
   - Monitoring configured on new server

1.13 Appendix A: File Permissions Reference

1.13.1 A.1 WordPress File Permissions Table

Proper file permissions are critical for security. WordPress files should not be world-writable.

Path	Type	Owner	Group	Mode	Purpose
/home/wordpress/public_html/	dir	www-data	www-data	755	Web root - allow read/execute
/home/wordp	file	www-data	www-data	644	PHP files - not world-writable
/home/wordpress/public_html/wp-config.php	file	www-data	www-data	600	Config - owner only readable
/home/wordp content/	dir	www-data	www-data	755	Content directory
/home/wordpress/public_html/wp-content/uploads/	dir	www-data	www-data	755	User uploads
/home/wordp content/upl	file	www-data	www-data	644	Uploaded files
/home/wordpress/public_html/wp-content/plugins/	dir	www-data	www-data	755	Plugins directory
/home/wordp content/the	dir	www-data	www-data	755	Themes directory
/home/wordpress/public_html/.htaccess	file	www-data	www-data	644	Rewrite rules
/home/wordp	dir	www-data	www-data	700	Backups - owner only

1.13.2 A.2 Permission Reset Script

Use this script to fix permissions after issues or new installations:

```
#!/bin/bash
# File: /usr/local/bin/wordpress-fix-permissions.sh
# Purpose: Reset WordPress file permissions to secure defaults

set -e

WP_ROOT="[CUSTOMIZE: /home/wordpress/public_html]"
WP_OWNER="www-data"
WP_GROUP="www-data"

if [ ! -d "$WP_ROOT" ]; then
    echo "Error: WordPress directory not found at $WP_ROOT"
    exit 1
fi
```

```
echo "Fixing WordPress permissions..."

# Directories: 755 (rwxr-xr-x)
echo "Setting directory permissions to 755..."
find "$WP_ROOT" -type d -exec chmod 755 {} \;

# Files: 644 (rw-r--r--)
echo "Setting file permissions to 644..."
find "$WP_ROOT" -type f -exec chmod 644 {} \;

# wp-config.php: 600 (rw-------)
echo "Setting wp-config.php to 600..."
chmod 600 "$WP_ROOT/wp-config.php"

# .htaccess: 644
echo "Setting .htaccess to 644..."
if [ -f "$WP_ROOT/.htaccess" ]; then
    chmod 644 "$WP_ROOT/.htaccess"
fi

# Ownership
echo "Setting ownership to $WP_OWNER:$WP_GROUP..."
chown -R "$WP_OWNER:$WP_GROUP" "$WP_ROOT"

# Backup directory: owner-only
echo "Setting backup directory permissions..."
if [ -d "$WP_ROOT/../backup" ]; then
    chmod 700 "$WP_ROOT/../backup"
    chmod 600 "$WP_ROOT/../backup"/*
fi

echo "▫ Permission reset complete"
echo "Verify with: ls -la $WP_ROOT"
```

Make executable:

```
sudo chmod +x /usr/local/bin/wordpress-fix-permissions.sh
```

Run when needed:

```
sudo /usr/local/bin/wordpress-fix-permissions.sh
```

1.14 Appendix B: wp-config.php Key Settings

1.14.1 B.1 Database Configuration

```
// Database configuration
define('DB_NAME', '[CUSTOMIZE: wordpress_db]');
define('DB_USER', '[CUSTOMIZE: wp_user]');
define('DB_PASSWORD', '[CUSTOMIZE: SecurePassword123!]');
define('DB_HOST', '[CUSTOMIZE: localhost]');
define('DB_CHARSET', 'utf8mb4');
define('DB_COLLATE', 'utf8mb4_unicode_ci');

// Optional: Non-standard port
define('DB_HOST', '[CUSTOMIZE: localhost:3307]');

// Optional: Socket connection
define('DB_HOST', '[CUSTOMIZE: localhost:/var/run/mysqld/mysqld.sock]');
```

1.14.2 B.2 Security Keys and Salts

Critical: Generate unique values using https://api.wordpress.org/secret-key/1.1/salt/

```
// Do not change these values! Generate new unique keys:
// Visit: https://api.wordpress.org/secret-key/1.1/salt/

define('AUTH_KEY',         'put your unique phrase here');
define('SECURE_AUTH_KEY',  'put your unique phrase here');
define('LOGGED_IN_KEY',    'put your unique phrase here');
define('NONCE_KEY',        'put your unique phrase here');
define('AUTH_SALT',        'put your unique phrase here');
define('SECURE_AUTH_SALT', 'put your unique phrase here');
define('LOGGED_IN_SALT',   'put your unique phrase here');
define('NONCE_SALT',       'put your unique phrase here');
```

> **WARNING:** Changing these keys will log out all users. Do this during maintenance window.

1.14.3 B.3 Security Constants

> **WARNING:** The `DISALLOW_FILE_MODS` constant prevents file editing through WordPress admin. When enabled, all updates must go through deployment processes.

```
// Disable file editing (REQUIRED in production)
define('DISALLOW_FILE_MODS', true);

// Disable plugin/theme installation
define('DISALLOW_FILE_MODS', true);  // Includes installation
```

```php
define('DISALLOW_PLUGIN_EDITING', true);  // If above is not used

// Disable plugin deactivation
define('DISALLOW_PLUGIN_ACTIVATION', true);

// Security: Disable XML-RPC (legacy remote publishing)
define('XMLRPC_REQUEST', false);

// Security: Force HTTPS
define('FORCE_SSL_ADMIN', true);
define('FORCE_SSL_LOGIN', true);

// Security: Cookie settings
define('SECURE_AUTH_COOKIE', true);  // Require HTTPS for auth cookie
define('LOGGED_IN_COOKIE', true);    // Require HTTPS for logged-in cookie
define('SECURE_LOGGED_IN_COOKIE', true);

// Security: Restrict REST API to authenticated users (optional)
// Requires custom code in functions.php to implement

// Security: Disable file editor in admin
define('DISALLOW_FILE_EDIT', true);  // No theme/plugin editor in wp-admin
```

1.14.4 B.4 Debugging Constants

```php
// Enable WordPress debugging (STAGING/DEV only, DISABLE in production)
define('WP_DEBUG', false);  // Set to true only for development
define('WP_DEBUG_LOG', true);  // Log errors to wp-content/debug.log
define('WP_DEBUG_DISPLAY', false);  // Don't display errors on frontend

// Script debugging
define('SCRIPT_DEBUG', false);  // Load unminified JS/CSS (dev only)

// Database debugging
define('SAVEQUERIES', false);  // Track database queries (dev only)
```

1.14.5 B.5 Performance Constants

```php
// Caching
define('WP_CACHE', true);  // Enable object caching

// Memory limits
define('WP_MEMORY_LIMIT', '256M');  // Per-request memory
define('WP_MAX_MEMORY_LIMIT', '512M');  // For background tasks

// Post revisions
```

```
define('WP_POST_REVISIONS', 5);  // Keep only 5 revisions per post

// Autosave interval (seconds)
define('AUTOSAVE_INTERVAL', 300);  // 5 minutes

// Trash retention (days)
define('EMPTY_TRASH_DAYS', 30);  // Auto-delete trash after 30 days

// Image quality
define('JPEG_QUALITY', 82);  // Compressed JPEG quality (0-100)

// Periodic backups
define('WP_AUTO_UPDATE_CORE', 'minor');  // Auto-update minor versions only
```

1.14.6 B.6 WordPress Cron Constants

```
// Disable WordPress cron (use system cron instead)
define('DISABLE_WP_CRON', true);

// If using system cron, set alternate cron endpoint
// This is handled via crontab entry:
# */5 * * * * curl -s https://example.com/wp-cron.php?doing_wp_cron > /dev/null 2>&1
```

1.14.7 B.7 Multisite Configuration (if applicable)

```
// Enable multisite (single blog)
define('WP_ALLOW_MULTISITE', true);

// Or if already multisite:
define('MULTISITE', true);
define('SUBDOMAIN_INSTALL', false);  // Use subfolders, not subdomains
define('DOMAIN_CURRENT_SITE', '[CUSTOMIZE: example.com]');
define('PATH_CURRENT_SITE', '/');
define('SITE_ID_CURRENT_SITE', 1);
define('BLOG_ID_CURRENT_SITE', 1);
```

1.14.8 B.8 Path Configuration (Optional)

```
// Override default paths (useful for non-standard installations)
define('WP_CONTENT_DIR', dirname(__FILE__) . '/wp-content');
define('WP_CONTENT_URL', 'https://[CUSTOMIZE: example.com]/wp-content');
define('WP_PLUGIN_DIR', WP_CONTENT_DIR . '/plugins');
define('WP_PLUGIN_URL', WP_CONTENT_URL . '/plugins');
```

1.14.9 B.9 Complete Recommended wp-config.php Template

```php
<?php
/**
 * WordPress Configuration File
 *
 * [CUSTOMIZE: Your site description]
 * Created: [DATE]
 */

// ============================================================
// DATABASE CONFIGURATION
// ============================================================
define('DB_NAME', '[CUSTOMIZE: wordpress_db]');
define('DB_USER', '[CUSTOMIZE: wp_user]');
define('DB_PASSWORD', '[CUSTOMIZE: SecurePassword123!]');
define('DB_HOST', 'localhost');
define('DB_CHARSET', 'utf8mb4');
define('DB_COLLATE', 'utf8mb4_unicode_ci');

// ============================================================
// SECURITY KEYS AND SALTS
// ============================================================
// Generate at: https://api.wordpress.org/secret-key/1.1/salt/
define('AUTH_KEY',         '[CUSTOMIZE: unique-key]');
define('SECURE_AUTH_KEY',  '[CUSTOMIZE: unique-key]');
define('LOGGED_IN_KEY',    '[CUSTOMIZE: unique-key]');
define('NONCE_KEY',        '[CUSTOMIZE: unique-key]');
define('AUTH_SALT',        '[CUSTOMIZE: unique-key]');
define('SECURE_AUTH_SALT', '[CUSTOMIZE: unique-key]');
define('LOGGED_IN_SALT',   '[CUSTOMIZE: unique-key]');
define('NONCE_SALT',       '[CUSTOMIZE: unique-key]');

// ============================================================
// DATABASE TABLE PREFIX
// ============================================================
$table_prefix = 'wp_';

// ============================================================
// LANGUAGE AND LOCALE
// ============================================================
define('WPLANG', 'en_US');

// ============================================================
// WORDPRESS SECURITY
// ============================================================
define('DISALLOW_FILE_MODS', true);  // Disable file editor
```

```php
define('FORCE_SSL_ADMIN', true);     // Require HTTPS for admin
define('FORCE_SSL_LOGIN', true);     // Require HTTPS for login
define('XMLRPC_REQUEST', false);     // Disable XML-RPC

// ============================================================
// PERFORMANCE & CACHING
// ============================================================
define('WP_CACHE', true);
define('WP_MEMORY_LIMIT', '256M');
define('WP_MAX_MEMORY_LIMIT', '512M');
define('WP_POST_REVISIONS', 5);
define('AUTOSAVE_INTERVAL', 300);
define('EMPTY_TRASH_DAYS', 30);

// ============================================================
// CRON & BACKGROUND TASKS
// ============================================================
define('DISABLE_WP_CRON', true);  // Use system cron

// ============================================================
// DEBUGGING (PRODUCTION = false)
// ============================================================
define('WP_DEBUG', false);
define('WP_DEBUG_LOG', true);
define('WP_DEBUG_DISPLAY', false);
define('SCRIPT_DEBUG', false);

// ============================================================
// WORDPRESS CORE UPDATES
// ============================================================
define('WP_AUTO_UPDATE_CORE', 'minor');  // Auto-update minor versions

// ============================================================
// WORDPRESS CORE PATHS
// ============================================================
if (!defined('ABSPATH')) {
    define('ABSPATH', dirname(__FILE__) . '/');
}

// ============================================================
// WORDPRESS CONFIGURATION LOADED - Load wp-settings.php
// ============================================================
require_once(ABSPATH . 'wp-settings.php');
?>
```

1.15 Appendix C: Change Log

1.15.1 Release History

This section documents all updates to this runbook and corresponding infrastructure changes.

1.15.1.1 Version 2.0 - February 2026

Major Updates: - Added comprehensive security hardening section (Section 5) - Expanded incident response procedures (Section 10) - Added disaster recovery procedures with RTO/RPO targets (Section 11) - Added automated monitoring and alerting section (Section 6.9) - Complete file permissions reference (Appendix A) - Detailed wp-config.php guidance (Appendix B)

Breaking Changes: - Enforced `DISALLOW_FILE_MODS = true` for all installations - Changed default post revision limit to 5 (from unlimited) - Updated PHP minimum version requirement to 8.1+

Infrastructure Changes: - Upgraded to MySQL 8.0 / MariaDB 10.6 - Added Redis caching layer - Implemented file integrity monitoring (AIDE) - Updated Nginx to 1.24+

Testing: - [CUSTOMIZE: Date] - Full disaster recovery test successful - [CUSTOMIZE: Date] - Security audit completed - [CUSTOMIZE: Date] - Performance testing done

1.15.1.2 Version 1.5 - August 2025

Updates: - Added database migration procedures (Section 8.2) - Expanded backup strategy and verification - Added WP-CLI command reference throughout - Updated SSL/TLS configuration for modern standards

Infrastructure: - SSL certificate renewal automated with Certbot - Added database backup redundancy (S3 + NAS) - Upgraded PHP to 8.0

1.15.1.3 Version 1.0 - February 2025

Initial Release: - Basic WordPress operations procedures - Deployment workflow - Backup and restore basics - Basic monitoring

---

1.16 Quick Reference Card (Printable)

For quick access during incidents, print this card and keep near your desk.

```
╔═══════════════════════════════════════════════════════════╗
║                                                           ╒
╒        WORDPRESS OPERATIONS QUICK REFERENCE               ╒
╒                                                           ╒
╚═══════════════════════════════════════════════════════════╝

EMERGENCY CONTACTS:
- SysAdmin On-Call: [CUSTOMIZE: PHONE]
```

```
- Database Admin: [CUSTOMIZE: PHONE]
- Security Officer: [CUSTOMIZE: PHONE]

CRITICAL COMMANDS:
Check if site is down:       curl -I https://example.com
View recent errors:          tail -30 /var/log/php-errors.log
Check server status:         top -bn1 | head
Disable all plugins:         wp plugin deactivate --all
Clear all caches:            wp cache flush && wp redis flush-db
Restore from backup:         wp db import backup/db_TIMESTAMP.sql

IMPORTANT PATHS:
WordPress root:              /home/wordpress/public_html/
Configuration:               /home/wordpress/public_html/wp-config.php
Backups:                     /home/wordpress/backup/
Logs:                        /var/log/nginx/, /var/log/php-errors.log
Database:                    MySQL [CUSTOMIZE: wordpress_db]

SERVICE COMMANDS:
Restart Nginx:               sudo systemctl restart nginx
Restart PHP-FPM:             sudo systemctl restart php8.1-fpm
Restart MySQL:               sudo systemctl restart mysql
Check service status:        sudo systemctl status [service]

ESCALATION PATH:
1. [CUSTOMIZE: First responder name]
2. [CUSTOMIZE: Manager name]
3. [CUSTOMIZE: Director name]
```

1.17 Related Documentation

External References: - WordPress Official Documentation - WordPress Security Handbook - WP-CLI Command Reference - Nginx Documentation - MySQL Documentation - OWASP Top 10

Internal References: - Network Infrastructure Documentation: [CUSTOMIZE: Link] - Security Policies: [CUSTOMIZE: Link] - Incident Management Process: [CUSTOMIZE: Link] - Change Management Process: [CUSTOMIZE: Link] - Disaster Recovery Plan: [CUSTOMIZE: Link]

1.18 Glossary

Term	Definition
LAMP/LEMP	Linux, Apache/Nginx, MySQL, PHP - server stack
WP-CLI	Command-line interface for WordPress management
wp-config.php	WordPress configuration file containing database credentials and settings
Transient	Temporary data stored in WordPress database or cache
Cron/WP-Cron	Scheduled background tasks
Plugin	Extension that adds functionality to WordPress
Theme	Collection of templates and assets defining site appearance
RTO	Recovery Time Objective - maximum acceptable downtime
RPO	Recovery Point Objective - maximum acceptable data loss
MTTR	Mean Time To Recovery - average time to recover from outage
FIM	File Integrity Monitoring - detecting unauthorized file changes
CDN	Content Delivery Network - geographically distributed servers
WebP	Modern image format with better compression than JPEG/PNG
HTTP Status Codes	200 (OK), 301 (Moved), 400 (Bad Request), 403 (Forbidden), 404 (Not Found), 500 (Server Error), 503 (Service Unavailable)

1.19 Document Metadata

Document Title: WordPress Operations Runbook
Version: 2.0
Status: Active
Last Updated: February 2026
Next Review Date: February 2027
Document Owner: [CUSTOMIZE: Operations Team]
Approval: [CUSTOMIZE: Manager Name and Date]

Change Request Process: 1. Submit change request to [CUSTOMIZE: owner email] 2. Include: section, change description, justification, testing done 3. Document will be updated and version incremented 4. All stakeholders notified of changes 5. Training scheduled if needed

Feedback & Improvements: To suggest improvements to this runbook, please contact [CUSTOMIZE: owner email] with: - What section or procedure - What was unclear or missing - Suggested improvement - Contact information

---

END OF DOCUMENT

This WordPress Operations Runbook is a comprehensive guide for managing WordPress installations in production environments. Regular review and updates are essential. All procedures should be tested in staging before implementation in production.